Ejercicio 03: corpus público y clasificador de texto reproducible

INF-8239 Ciencia de Datos II, Unidad 02 (U02.E03)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RoqueVentura/UASD-Maestria-Ciencia-De-Datos, Ciencia de datos II/Unidad II/Ejercicio 03
Evidencias	reports/EvidenciaLab04_Lab05.pdf, capturas de la ejecución de LAB04 y LAB05

1. Corpus, licencia y pregunta

Trabajé con *Twitter Financial News Sentiment*, publicado en Hugging Face por zeroshot (commit ccbe24d, 23 de febrero de 2024). Contiene titulares y tuits financieros en inglés anotados con tres clases: 0 = Bearish, 1 = Bullish y 2 = Neutral. Tiene licencia MIT, que permite descargar, analizar y publicar resultados. Uso el archivo `sent_train.csv` (9 543 textos), que se descarga por enlace directo con `scripts/download_data.py` configurando `DATASET_URL` en `.env`. El SHA-256 del archivo (71aa99d0…c5420e8) permite verificar que otra persona obtiene la misma versión.

La pregunta es si se puede predecir el sentimiento de mercado de un titular financiero a partir de su texto. El error más costoso es no detectar una noticia negativa (Bearish) y presentarla como neutral. También evalué *fake_news_corpus_spanish*, que descarté:

Criterio	Twitter Financial News Sentiment (seleccionado)	fake_news_corpus_spanish
Licencia	MIT	CC BY 4.0
Tamaño	9 543 textos (train), 2 388 en validación	572 textos en un único split "test"
Etiqueta	3 clases documentadas en la ficha	CATEGORY (true/false) sin definición en la ficha
Problemas	Solo inglés y dominio financiero; clases desbalanceadas	Tamaño insuficiente; la columna SOURCE puede delatar la etiqueta

2. Auditoría, duplicados y prevención de fuga

La auditoría (`scripts/audit_data.py`, salida en `reports/audit_data.txt`) no encontró nulos ni textos duplicados exactos. La distribución está desbalanceada: Neutral 64.7 %, Bullish 20.2 % y Bearish 15.1 %, por lo que la accuracy aislada sería engañosa. La longitud de los textos es p50 = 79, p90 = 139 y p99 = 140 caracteres, sin textos vacíos. La ficha y el diccionario están en `docs/DATASET_CARD.md`.

Para evitar fuga, el script de entrenamiento elimina duplicados de texto **antes** de la partición y el vectorizador TF-IDF vive dentro de cada pipeline, por lo que solo aprende el vocabulario del entrenamiento. El contrato de datos (`validate_dataframe`) detiene el proceso si faltan columnas, hay textos vacíos o existe una sola clase.

3. Baseline y pipelines comparables

Los tres modelos comparten la misma partición estratificada 75/25 con semilla 42 (2 386 textos de prueba) y el mismo vectorizador TF-IDF de unigramas y bigramas con *sublinear_tf*:

Modelo (conjunto de prueba)	F1 macro	Accuracy	Recall Bearish	Recall Bullish	Recall Neutral
Dummy, clase más frecuente (baseline)	0.262	0.648	0.000	0.000	1.000
Complement Naive Bayes	0.658	0.779	0.347	0.541	0.954
Regresión logística, class_weight=balanced	**0.726**	**0.795**	0.611	0.713	0.863

El baseline obtiene 0.648 de accuracy sin aprender nada; por eso se compara con F1 macro.

Ambos clasificadores superan ampliamente al baseline. La regresión logística mejora a Naive Bayes en 0.07 de F1 macro, y la diferencia está en las clases minoritarias: Naive Bayes solo recupera el 35 % de las noticias Bearish, frente al 61 % de la regresión logística. Por eso seleccioné la regresión logística y la guardé en `models/text_model.joblib`.

4. Evaluación del modelo seleccionado

Clase	Precision	Recall	F1	Support
0 · Bearish	0.606	0.611	0.609	360
1 · Bullish	0.686	0.713	0.699	481
2 · Neutral	0.875	0.863	0.869	1 545

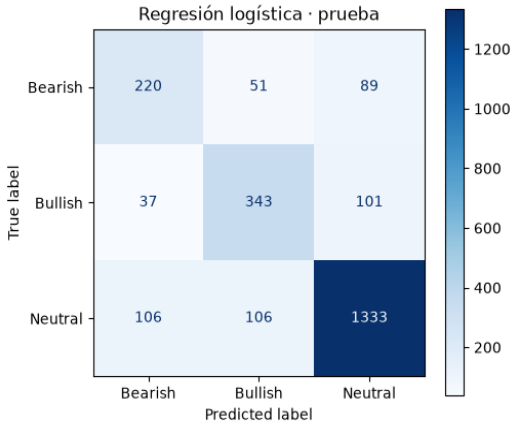


Figura 1. Matriz de confusión de la regresión logística en prueba (notebook, sección 8).



Figura 2. Aplicación Streamlit local: predicción 1 (Bullish) con advertencia de alcance.

Bearish es la clase más difícil: tiene el menor recall (0.611) y la menor precision (0.606). De 360 noticias negativas, 140 se pierden y 89 de ellas se clasifican como Neutral. Las confusiones más frecuentes son noticias neutrales marcadas como Bearish o como Bullish (106 cada una). En este contexto, una noticia negativa leída como neutral oculta un riesgo de mercado, y una neutral leída como Bearish o Bullish es una falsa señal que podría motivar a comprar o vender sin fundamento.

5. Análisis de 20 errores

El modelo cometió 490 errores en prueba. Clasifiqué manualmente 20 en la columna `category` de `reports/error_analysis.csv`, sin modificar ninguna predicción:

Categoría	Errores	Ejemplo (real, predicho)
Jerga financiera y cifras	8	"Celanese Q4 profits almost halved" (Bearish, Neutral)
Ambigüedad	4	"Asian shares mixed as doubts grow on elusive U.S.-China trade deal" (Bearish, Bullish)
Etiqueta discutible	3	"Jokowi Warns of Economic Calamity From Forest Fires" (Neutral, Bearish)
Tema fuera del dominio	3	"It is only failure if you stay down after the fall…" (Neutral, Bearish)
Texto insuficiente	1	"Ahold Delhaize share buyback update" (Neutral, Bullish)
Negación	1	"Nokia's surprise profit rise fails to salvage 2019 dividend" (Bearish, Bullish)

La categoría dominante permite formular una hipótesis verificable: TF-IDF no capta que expresiones como "+14.6%", "halved", "puts" o "hits the sidelines" llevan sentimiento, y el signo de cifras como "-3.5%" se pierde al tokenizar. El siguiente experimento es conservar las cifras con su signo o incorporar un léxico o modelo del dominio financiero, y comparar el recall de Bearish.

6. Código, repositorio y reproducibilidad

El proyecto usa uv con Python 3.12.13 y scikit-learn 1.9.1; `uv.lock` fija las versiones. El código reutilizable está en `src/inf8239_u02`: `config.py` lee la configuración de `.env`, `data.py` carga, valida y calcula el SHA-256, y `modeling.py` construye los tres pipelines. El README explica cómo obtener el dataset, entrenar, ejecutar el notebook y las pruebas y abrir la aplicación. Ni el dataset ni `.env` se suben a Git, y no hay rutas absolutas ni credenciales en el código.

El notebook `notebooks/01_corpus_clasificador_texto.ipynb` está ejecutado e integra ambos laboratorios: procedencia y SHA-256, auditoría, contrato, duplicados y fuga, entrenamiento, métricas, matriz, los 20 errores e inferencia. Reproduce las métricas de `reports/text_metrics.csv` y comprueba que el modelo guardado da las mismas predicciones que el reentrenado. `requirements.txt`, generado con `uv export`, permite instalar las mismas versiones con pip.

Paso	Comando
Entorno	uv sync (o pip install -r requirements.txt)
Datos	uv run python scripts/download_data.py · uv run python scripts/audit_data.py
Entrenamiento	uv run python scripts/train_text.py
Notebook	uv run jupyter execute --inplace notebooks/01_corpus_clasificador_texto.ipynb
Pruebas	uv run pytest -q
Aplicación	uv run streamlit run app/streamlit_app.py
Nube	requirements-cloud.txt (uv export); Colab: requirements-colab.txt

Los resultados están en `reports/`: text_metrics.csv, confusion_text.png, error_analysis.csv y model_evaluation.md. El cierre interpretativo y la conclusión están en `docs/cierre_lab05.md`.

7. Pruebas automatizadas

Archivo	Prueba	Qué comprueba
test_data_contract.py	test_accepts_valid_dataframe	Un DataFrame con texto y etiqueta válidos pasa la validación
test_data_contract.py	test_rejects_missing_target	Se rechaza un dataset sin la columna objetivo
test_data_contract.py	test_rejects_empty_text	Se rechazan textos vacíos
test_text_model.py	test_each_pipeline_predicts_one_label	Los tres pipelines entrenan, predicen y conservan tfidf y model
test_embeddings_network.py	test_tokenize_normalizes_and_removes_one_character_tokens	El tokenizador normaliza y descarta tokens de un carácter
test_embeddings_network.py	test_demo_network_has_valid_pagerank	PageRank de la red de demostración es válido

Resultado: las 6 pruebas pasan (6 passed, pytest 8.4.2; salida en reports/pytest_lab05.txt). Las pruebas verifican las reglas programadas, no la representatividad del corpus.

8. Conclusión

El corpus Twitter Financial News Sentiment cumple los criterios de selección: licencia MIT, acceso reproducible por URL con hash verificable, etiqueta interpretable y 9 543 observaciones sin nulos ni duplicados exactos. Su principal limitación es el desbalance: el 65 % de los textos es Neutral, lo que permite a un baseline trivial alcanzar 0.648 de accuracy. Con F1 macro como métrica principal, la regresión logística (0.726) supera a Complement Naive Bayes (0.658) y al baseline (0.262).

El modelo aún no es fiable para la clase Bearish: pierde casi 4 de cada 10 noticias negativas, la mayoría confundidas con Neutral. El análisis de errores apunta a la jerga financiera y a las cifras como causa principal, lo

que define el siguiente experimento. La aplicación Streamlit responde bien a titulares claros, pero ante textos ambiguos o fuera del dominio igual devuelve una clase, por lo que su salida es una predicción condicionada por los datos y no una verdad.

Las conclusiones se limitan a titulares financieros en inglés publicados en Twitter, mayormente del mercado estadounidense, en un periodo que el publicador no documenta. Antes de un uso real habría que mejorar el recall de Bearish y validar con `sent_valid.csv`. El trabajo es académico y no constituye una recomendación de inversión.

9. Uso de inteligencia artificial

Herramienta	Claude Code (modelo Claude Opus 5.5, de Anthropic), usado desde Visual Studio Code en octubre de 2026.
Uso	Apoyo para aclarar los pasos de los laboratorios, explicar errores de ejecución, revisar el cumplimiento del mandato y mejorar la redacción.
Prompts relevantes	"Ponme los pasos organizados"; "Genérame un comando que haga esto" para el paso 4; "Verifica qué me faltó de acuerdo al mandato del maestro"; "Verifica bien y crea el ipynb". Este último se debe a que las guías de LAB04 y LAB05 no incluyen un notebook: ejecuté los laboratorios paso a paso con los scripts y guardé cada captura en EvidenciaLab04_Lab05.pdf. Como el mandato del Ejercicio 03 solicita un notebook ejecutado, pedí a la IA crearlo a partir de esos mismos scripts, sin cambiar los resultados.
Partes asistidas	Comando de comparación de candidatos, borradores de la ficha, del cierre, de la conclusión y de este informe, propuesta de categorías para los 20 errores, y creación y ejecución del notebook integrador.
Trabajo propio	Comparé y aprobé el corpus, configuré .env, ejecuté paso a paso la descarga, la auditoría, el entrenamiento y las pruebas, probé la aplicación con mis propios textos, armé el documento de evidencias con las capturas e hice los commits.
Verificación	Las cifras provienen de ejecutar los scripts en mi entorno: el SHA-256 coincide en descarga y auditoría, y las métricas coinciden con reports/text_metrics.csv y con la salida de train_text.py. Las 6 pruebas pasan.
Correcciones	Detecté que el comando del paso 4 fallaba en PowerShell y pedí corregirlo. Exigí que la ficha conservara el formato de la plantilla y retiré del notebook y del informe análisis, citas y sugerencias que el mandato no pedía. También corregí git_log.txt (UTF-16), retiré de Git archivos temporales de Word y las tildes del notebook en Windows (PYTHONUTF8=1).
Responsabilidad	Revisé el contenido final, puedo explicar cada parte y asumo la responsabilidad de lo entregado.

10. Fuentes

zeroshot. (2022). *Twitter Financial News Sentiment* [Dataset]. Hugging Face. https://huggingface.co/datasets/zeroshot/twitter-financial-news-sentiment

Grandury, M. (2024). *fake_news_corpus_spanish* [Dataset]. Hugging Face. https://huggingface.co/datasets/mariagrandury/fake_news_corpus_spanish